Parallel Mandelbrot - Report

Exercise 2: Data Parallelism

1. Approach

The Mandelbrot set was implemented in C using OpenMP for parallelization. The stb_image_write library was used for PNG image output. The project runs inside a VS Code Dev Container (Ubuntu 22.04) to ensure consistent builds across macOS, Windows, and Linux.

Parallelization Strategy:

We used data parallelism via OpenMP's `#pragma omp parallel for` directive on the outer loop (rows of pixels). Each thread processes a set of rows independently. Since each pixel's computation is completely independent from every other pixel, no synchronization or locking is needed — there are no shared variables being written to, and each thread writes to its own section of the image array.

Scheduling Scheme:

We chose `schedule(dynamic)` instead of the default static scheduling. This is important because not all pixels require the same amount of work. Pixels outside the Mandelbrot set diverge quickly (sometimes after just 2 iterations), while pixels inside or near the boundary need the full 1000 iterations. With static scheduling, some threads would finish early and sit idle while others are still computing the complex boundary regions. Dynamic scheduling distributes rows on-demand, so threads that finish early immediately pick up new work.

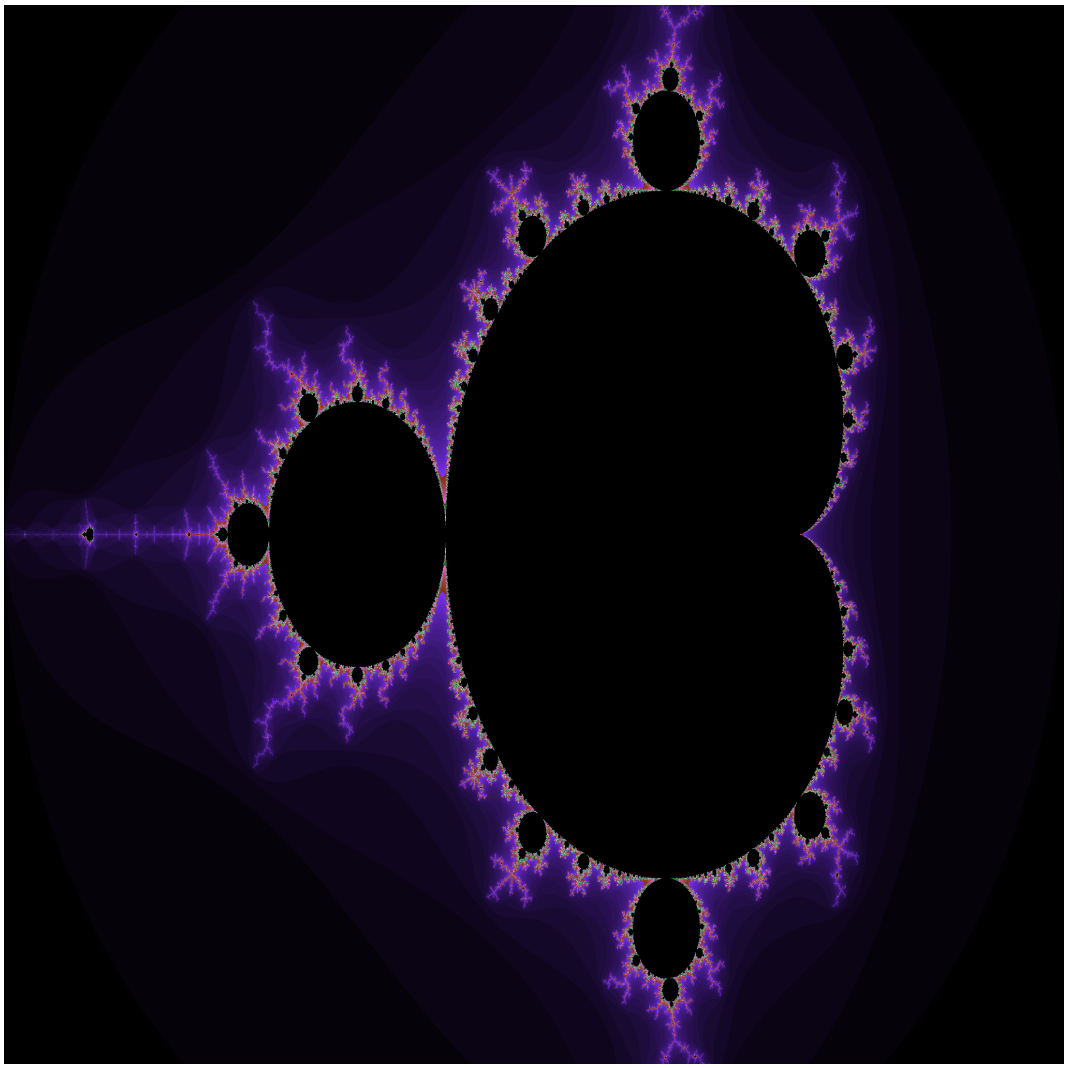
The program accepts configurable parameters via command line arguments: image dimensions (width, height), maximum iterations, thread count (for benchmarking), and viewport coordinates (min_x, min_y, max_x, max_y) for zooming.

2. Example Image

The following image was generated with the command:

```
./mandelbrot 2048 2048 1000 8
```

Settings: 2048x2048 pixels, 1000 max iterations, 8 threads, Viewport: (-2.0, -1.0, 1.0, 1.0)



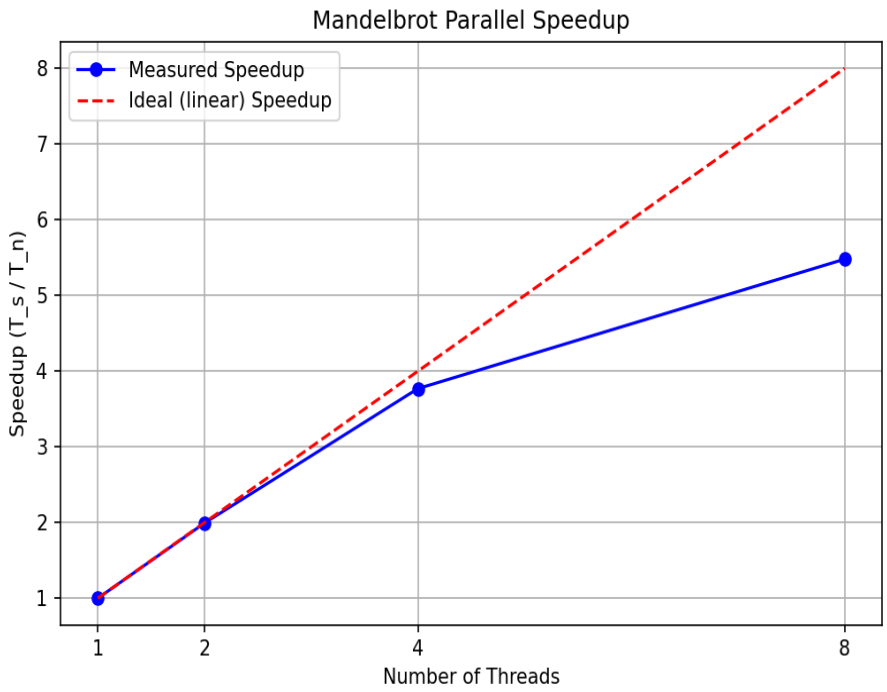
3. Performance Measurements

All benchmarks were run on a 2048x2048 image with 1000 max iterations. Only the pure computation time was measured (excluding disk I/O), using `omp_get_wtime()` before and after the parallel loop. The Docker container had access to 8 CPU cores.

Threads	Time (s)	Speedup (T_s / T_n)
1	3.7314	1.00x
2	1.8722	1.99x
4	0.9900	3.77x
8	0.6810	5.48x

We also ran additional tests beyond the core count to observe the effect of over-subscribing threads:

Threads	Time (s)	Speedup (T_s / T_n)
16	0.6407	5.83x
32	0.6435	5.80x
64	0.6493	5.75x



4. Analysis

The results show near-linear speedup up to 2 threads (1.99x), with diminishing returns as thread count increases. At 4 threads we achieved 3.77x speedup (vs. ideal 4.0x), and at 8 threads we reached 5.48x (vs. ideal 8.0x).

Beyond 8 threads (the number of available CPU cores), performance completely plateaus. Running with 16, 32, or 64 threads gives no improvement because the extra threads simply queue up waiting for a free core. There is no additional parallelism to exploit, and the small overhead of managing more threads causes a very slight slowdown.

The gap between measured and ideal speedup is expected and can be attributed to:

- Thread creation and scheduling overhead
- Memory bandwidth becoming a bottleneck as all cores access the image array
- Cache contention between cores
- The inherent serial portion of the program (Amdahl's Law)

The choice of `schedule(dynamic)` was important for load balancing. Pixels near the boundary of the Mandelbrot set require significantly more iterations than those far outside the set, so static scheduling would lead to uneven workloads across threads.